102. Перегрузка функций

На этом уроке мы рассмотрим перегрузку функций в языке C++, что это такое и как её эффективно использовать.

Оглавление:

1. Перегрузка функций
2. Типы возврата в перегрузке функций
3. Псевдонимы типов в перегрузке функций
4. Вызовы функций
5. Несколько совпадений
6. Заключение

Перегрузка функций

Перегрузка функций — это возможность определять несколько функций с одним и тем же именем, но с разными параметрами. Например:

```
int subtract(int a, int b)
{
    return a - b;
}
```

Здесь мы выполняем операцию вычитания с целыми числами. Однако, что, если нам нужно использовать числа **типа с плавающей запятой**? Эта функция совсем не подходит, так как любые параметры типа double будут конвертироваться в **тип int**, в результате чего будет теряться дробная часть значений.

Одним из способов решения этой проблемы является определение двух функций с разными именами и параметрами:

```
int subtractInteger(int a, int b)
{
    return a - b;
}

double subtractDouble(double a, double b)
{
    return a - b;
}
```

Но есть и лучшее решение — перегрузка функции. Мы можем просто определить еще одну функцию subtract(), которая принимает параметры типа double:

```
double subtract(double a, double b)
{
    return a - b;
}
```

Теперь у нас есть две версии функции subtract():

```
int subtract(int a, int b); // целочисленная версия
double subtract(double a, double b); // версия типа с плавающей запятой
```

Может показаться, что произойдет **конфликт имен**, но это не так. Компилятор может определить сам, какую версию subtract() следует вызывать на основе аргументов, используемых в вызове функции. Если параметрами будут переменные типа int, то C++ понимает, что мы хотим вызвать `subtract(int, int)`. Если же мы предоставим два значения типа с плавающей запятой, то C++ поймет, что мы хотим вызвать `subtract(double, double)`. Фактически, мы можем определить столько перегруженных функций subtract(), сколько хотим, до тех пор, пока каждая из них будет иметь свои (уникальные) параметры.

Следовательно, можно определить функцию subtract() и с большим количеством параметров:

```
int subtract(int a, int b, int c)
{
    return a - b - c;
}
```

Хотя здесь subtract() имеет 3 параметра вместо 2, это не является ошибкой, поскольку эти параметры отличаются от параметров других версий subtract().

Типы возврата в перегрузке функций

Обратите внимание, **тип возврата функции** НЕ учитывается при перегрузке функции. Предположим, что вы хотите написать функцию, которая возвращает **рандомное число**, но вам нужна одна версия, которая возвращает значение типа int, и вторая — которая возвращает значение типа double. У вас может возникнуть соблазн сделать следующее:

```
int getRandomValue();
double getRandomValue();
```

Компилятор выдаст ошибку. Эти две функции имеют одинаковые параметры (точнее, они отсутствуют), и, следовательно, второй вызов функции getRandomValue() будет рассматриваться как ошибочное переопределение первого вызова. Имена функций нужно будет изменить.

Псевдонимы типов в перегрузке функций

Поскольку объявление **typedef** (псевдонима типа) не создает новый тип данных, то следующие два объявления функции print() считаются идентичными:

```
typedef char *string;
void print(string value);
void print(char *value);
```

Вызовы функций

Выполнение вызова перегруженной функции приводит к одному из трех возможных результатов:

- **Совпадение найдено.** Вызов разрешен для соответствующей перегруженной функции.
- **Совпадение не найдено.** Аргументы не соответствуют любой из перегруженных функций.
- **Найдены несколько совпадений.** Аргументы соответствуют более чем одной перегруженной функции.

При компиляции перегруженной функции, C++ выполняет следующие шаги для определения того, какую версию функции следует вызывать:

Шаг №1: **C++ пытается найти точное совпадение**. Это тот случай, когда фактический аргумент точно соответствует типу параметра одной из перегруженных функций. Например:

```
void print(char *value);
void print(int value);

print(0); // точное совпадение с print(int)
```

Хотя `0` может технически соответствовать и `print(char *)` (как **нулевой указатель**), но он точно соответствует `print(int)`. Таким образом, `print(int)` является лучшим (точным) совпадением.

Шаг №2: **Если точного совпадения не найдено, то C++ пытается найти совпадение путем дальнейшего неявного преобразования типов**. Ранее мы говорили о том, как определенные типы данных могут автоматически конвертироваться в другие типы данных. Если вкратце, то:

- char, unsigned char и short конвертируются в int;
- unsigned short может конвертироваться в int или unsigned int (в зависимости от размера int);
- float конвертируется в double;
- **enum** конвертируется в int.

Например:

```
void print(char *value);
void print(int value);

print('b'); // совпадение с print(int) после неявного преобразования
```

В этом случае, поскольку нет `print(char)`, символ `b` конвертируется в тип int, который затем уже соответствует `print(int)`.

Шаг №3: **Если неявное преобразование невозможно, то C++ пытается найти соответствие посредством стандартного преобразования**. В стандартном преобразовании:

- Любой числовой тип будет соответствовать любому другому числовому типу, включая unsigned (например, int равно float).
- enum соответствует формальному типу числового типа данных (например, enum равно float).
- Ноль соответствует типу указателя и числовому типу (например, `0` как `char *` или `0` как `float`).
- Указатель соответствует **указателю типа void**.

Например:

```
struct Employee; // определение опустим
void print(float value);
void print(Employee value);

print('b'); // 'b' конвертируется в соответствие версии print(float)
```

В этом случае, поскольку нет `print(char)` (точного совпадения) и нет `print(int)` (совпадения путем неявного преобразования), символ `b` конвертируется в тип float и сопоставляется с `print(float)`.

Обратите внимание, все стандартные преобразования считаются равными. Ни одно из них не считается выше остальных по приоритету.

Шаг №4: **C++ пытается найти соответствие путем пользовательского преобразования**. Хотя мы еще не рассматривали классы, но они могут определять преобразования в другие типы данных, которые могут быть неявно применены к объектам этих классов. Например, мы можем создать класс `W` и в нем определить пользовательское преобразование в тип int:

```
class W; // с пользовательским преобразованием в тип int

void print(float value);
void print(int value);

class W value; // объявлен переменную value типа класса W
print(value); // value конвертируется в int по, следовательно, соответствует print(int)
```

Хотя `value` относится к типу класса `W`, но, поскольку тот имеет пользовательское преобразование в тип int, вызов `print(value)` соответствует версии `print(int)`.

То, как делать пользовательские преобразования в классах, мы рассмотрим на соответствующих уроках.

Несколько совпадений

Если каждая из перегруженных функций должна иметь уникальные параметры, то как могут быть возможны несколько совпадений? Поскольку все стандартные и пользовательские преобразования считаются равными, то, если вызов функции соответствует нескольким кандидатам посредством стандартного или пользовательского преобразования, результатом будет **неоднозначное совпадение** (т.е. несколько совпадений). Например:

```
void print(unsigned int value);
void print(float value);

print('b');
print(0);
print(3.14159);
```

В случае с `print('b')` C++ не может найти точного совпадения. Он пытается преобразовать `b` в тип int, но версии `print(int)` тоже нет. Используя стандартное преобразование, C++ может преобразовать `b` как в unsigned int, так и во float. Поскольку все стандартные преобразования считаются равными, то получается два совпадения.

С `print(0)` всё аналогично. `0` — это int, а версии `print(int)` нет. Путем стандартного преобразования мы опять получаем два совпадения.

А вот с `print(3.14159)` всё несколько запутаннее: большинство программистов отнесут его однозначно к `print(float)`. Однако, помните, что по умолчанию все значения-литералы типа с плавающей запятой относятся к типу double, если у них нет окончания `f`. `3.14159` — это значение типа double, а версии `print(double)` нет. Следовательно, мы получаем ту же ситуацию, что и в предыдущих случаях — неоднозначное совпадение (два варианта).

Неоднозначное совпадение считается ошибкой типа compile-time. Следовательно, оно должно быть устранено до того, как ваша программа скомпилируется. Есть два решения этой проблемы:

Решение №1: Просто определить новую перегруженную функцию, которая принимает параметры именно того типа данных, который вы используете в вызове функции. Тогда C++ сможет найти точное совпадение.

Решение №2: Явно преобразовать с помощью **операторов явного преобразования** неоднозначный параметр(ы) в соответствии с типом функции, которую вы хотите вызвать. Например, чтобы вызов `print(0)` соответствовал `print(unsigned int)`, вам нужно сделать следующее:

```
print(static_cast<unsigned int>(0)); // производит вызов print(unsigned int)
```

Заключение

Перегрузка функций может значительно снизить сложность программы, в то же время создавая небольшой дополнительный риск. Хотя этот урок несколько долгий и может показаться сложным, но, на самом деле, перегрузка функций обычно работает прозрачно и без каких-либо проблем. Все неоднозначные случаи компилятор будет отмечать, и их можно будет легко исправить.

Правило: Используйте перегрузку функций для упрощения ваших программ.